Turnly — Hospital Token & Queue Management App

Concise Project Plan (Flutter + Firebase)

Core Idea

IMPORTANT

Patients waste hours waiting in hospital OPD queues. Turnly lets them book a digital token from home, track their position in the queue live, and get notified when their turn is near. Hospitals register on the platform, and an Admin approves them before they go live. Everything runs inside a single Flutter mobile app — no web panel needed.

App Flow (Concise Workflow)



Three Roles in the App

Role	What They Do	Screens
Patient (User)	Sign up, search hospitals, book token, track queue, get notifications	Login, Home, Hospital Detail, Booking, My Tokens, Profile
Manager (Hospital)	Register hospital, manage departments/doctors, run the queue	Registration Form, Dashboard, Doctor Management, Queue Control
Admin (Platform)	Approve/reject hospitals, view stats, suspend hospitals	Admin Panel (approval list, stats) — all inside the same mobile app

Features — Implementation Order (Priority Sequence)

Phase 1: Foundation (Days 1–2)

#	Feature	What to Build	Tech Stack
F1	Project Setup & Data Model	Firebase project, Flutter project, Firestore collections with hospitalId on every document	Flutter, Firebase Core, Cloud Firestore
F2	Auth & Role Selection	Splash → "User or Manager" choice. Email/password sign-up & login. Role stored in Firestore users collection	Firebase Auth, Firestore

Phase 2: Patient App — Core (Days 3–6)

#	Feature	What to Build	Tech Stack
F3	Hospital Browsing	List all approved hospitals, search by name/city, hospital detail page	Firestore queries, Flutter ListView/SearchBar
F3b	Departments & Doctors	Show departments of selected hospital → doctors in each department with availability	Firestore sub-collections
F4	Token Booking	Book a token for self or family member. Cloud Function assigns next token number safely. Daily limit check	Cloud Functions (Node.js/TypeScript), Firestore Transactions
F5	Live Queue Tracking	"My Token" screen — shows position, status (Waiting → Serving → Done), live updates	Firestore real-time snapshots (StreamBuilder)

Phase 3: Manager Side (Days 2–5, parallel)

#	Feature	What to Build	Tech Stack
F7	Hospital Registration Form	In-app multi-step form: hospital name, type, city, address, phone, OPD timings, manager details → saved as status: pending	Firestore, Flutter Forms
F8	Manager Dashboard	In-app dashboard after approval: add/edit departments, doctors, OPD hours, token limits	Firestore CRUD
F8b	Queue Management	In-app per-doctor queue view. "Call Next", "Skip No-Show", "Mark Complete" buttons	Firestore real-time updates
F8c	Priority Tokens	Mark elderly/disabled/pregnant/urgent → they move up in queue	Firestore ordering with priority field

Phase 4: Admin Panel (Day 10)

#	Feature	What to Build	Tech Stack
F9	Hospital Approval	In-app admin screen: list pending registrations. Approve / Reject / Suspend buttons	Firestore queries + Cloud Functions
F9b	Platform Stats	In-app stats screen: total hospitals, tokens issued per hospital, active queues	Firestore aggregation queries

Phase 5: Notifications & Smart Features (Days 6, 8–9)

#	Feature	What to Build	Tech Stack
F10	Push Notifications	Alert when 2–3 patients ahead + when doctor calls your token	Firebase Cloud Messaging (FCM) + Cloud Functions
F11	Waiting Time Estimate	Formula: avg consultation time × patients ahead	Cloud Functions, Firestore

Phase 6: Polish & Extras (Days 8–12)

#	Feature	What to Build	Tech Stack
F6	Cancel & History	Cancel active token, view past tokens	Firestore queries
F6b	Family Members	Save family profiles, book tokens on their behalf	Firestore sub-collection under user
F6c	Search & Filters	Search doctors by name, filter by department	Firestore queries + Flutter search
F12	AI Department Guide (Optional)	Chat helper suggests correct OPD department based on symptoms	Gemini API / any LLM via Cloud Function

Phase 7: Security & Release (Days 12–15)

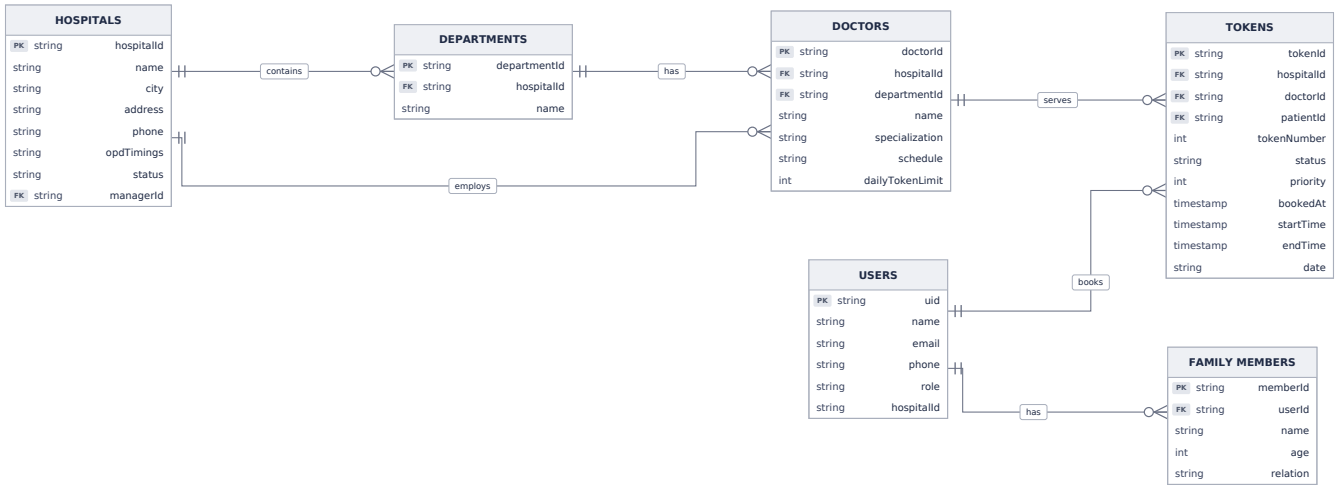
#	Feature	What to Build	Tech Stack
F13	Security Rules	Firestore rules: staff sees only own hospital, patients see only own tokens	Firestore Security Rules
F14	Testing & Release	End-to-end test, build APK, demo with 2 hospitals	Flutter build, APK release

Complete Tech Stack

Layer	Technology	Purpose
Frontend	Flutter (Dart)	Single mobile app for all roles (Android + iOS)
Backend	Firebase	Full serverless backend
Auth	Firebase Authentication	Email/password login (+ phone OTP if needed)
Database	Cloud Firestore	Real-time NoSQL database with hospitalId scoping
Server Logic	Firebase Cloud Functions (Node.js/TypeScript)	Token numbering, notifications, waiting time calc
Push Notifications	Firebase Cloud Messaging (FCM)	Turn-is-near alerts, token called alerts
Storage	Firebase Storage	Hospital logos, profile photos

Layer	Technology	Purpose
Architecture	MVVM (Model-View-ViewModel)	View → ViewModel → Model, with Services talking to Firebase
State Management	GetX	Flutter app state management (ViewModels, reactive state, routing)
Dependency Injection	GetX (Bindings)	Connect Services and ViewModels to the Views (Get.put / Bindings)
Local Notifications	flutter_local_notifications	Show notification UI on device
AI (Optional)	Gemini API or Google AI Dart SDK	AI department suggestion chatbot

Firestore Data Model (Collections)



Key Flutter Packages

Package	Purpose
firebase_core	Firebase initialization
firebase_auth	Authentication
cloud_firestore	Database
firebase_messaging	Push notifications (FCM)
firebase_cloud_functions	Call Cloud Functions
firebase_storage	Image uploads
flutter_local_notifications	Local notification display
get	State management, dependency injection and routing (GetX)
google_fonts	Typography
intl	Date/time formatting
cached_network_image	Image caching
shimmer	Loading placeholders

Screen Map

Push Notification Strategy

> **TIP**
> Firebase Cloud Messaging (FCM) is the recommended and free solution for push notifications with Flutter + Firebase.

Trigger	Notification	How It Works
2–3 patients ahead	"Your turn is coming! Head to the hospital"	Cloud Function watches queue changes in Firestore → sends FCM to patient's device token
Doctor calls your token	"You're being called! Go to Room X"	Staff clicks "Call Next" → Cloud Function triggers FCM
Token booked successfully	"Token #12 booked for Dr. Ahmed at City Hospital"	Immediate FCM on booking
Hospital approved	"Your hospital has been approved!"	Admin action triggers FCM to manager

Stack for notifications:

- firebase_messaging (Flutter package) — receive FCM on device
- flutter_local_notifications — display notification UI
- Cloud Functions (TypeScript) — server-side trigger that sends FCM messages
- Firestore fcmTokens — store each user's device token on login

⚡ What to Build First → Last (Exact Order)

> **IMPORTANT**
> Follow this exact sequence. Each step depends on the one before it.

Step	What	Why First
1	Firebase + Flutter project setup, data model	Everything depends on this
2	Auth (Login/Signup) + Role selection screen	Can't do anything without users
3	Hospital Registration Form (Manager)	Need hospitals in the system
4	Admin Approval Panel	Need to approve hospitals to make them visible
5	Manager Dashboard (add doctors, departments)	Approved hospitals need content
6	Patient Home — Browse/Search Hospitals	Patients need to see hospitals
7	Department & Doctor listing	Patient needs to pick a doctor
8	Token Booking (Cloud Function)	Core feature — the whole point of the app
9	Live Queue Tracking	Patient needs to see their position
10	Queue Management (Call Next, Skip, Complete)	Staff needs to move the queue
11	Push Notifications (FCM)	Turn-is-near alerts
12	Cancel, History, Family Members	Polish features
13	Waiting Time Estimate	Nice-to-have
14	Security Rules	Lock down data
15	AI Department Guide (Optional)	Only if time allows

What's Removed / Simplified from Original Document

Original	Change	Reason
Separate web dashboards	Single Flutter mobile app with role-based screens	One codebase, one app
Phone OTP login	Email/password only (add OTP later)	Faster to implement
Staff sub-accounts	Manager is the only hospital user	Simplify for MVP

> **NOTE**
> This is a 100% mobile app — no web panel at all. Admin, Manager, and Patient all use the same Flutter app with different screens based on their role.

MVP Definition (Minimum to Demo)

Your demo should show this complete flow:

1. Manager opens app → selects "Manager" → fills registration form
2. Admin logs in → sees pending hospital → approves it
3. Manager gets dashboard → adds department + doctor with timings
4. Patient opens app → selects "User" → signs up
5. Patient searches hospitals → selects the approved one → sees doctors
6. Patient books a token → gets token #1
7. Patient sees live queue position on "My Token" screen
8. Manager clicks "Call Next" → patient gets notification
9. Patient token status changes to "Serving" → then "Done"

That's the full loop. Everything else is polish.

Architecture Decision: MVVM vs Clean Architecture

Chosen: MVVM (Model-View-ViewModel)

Comparison

Criteria	MVVM	Clean Architecture
Layers	3 — View → ViewModel → Model	5+ — Presentation → Domain → Data (each with sub-layers)
Boilerplate	Minimal	Heavy (Use Cases, Repositories, Data Sources, Entities, Mappers)
Setup time	~1 day	3–4 days just for folder structure and abstractions
Learning curve	Simple, beginner-friendly	Complex, requires deep understanding of SOLID principles
Firebase fit	Perfect — Firebase IS the data layer, MVVM wraps it directly	Forces unnecessary abstractions over Firebase
Team of 2	Easy to divide and understand	Too many layers = merge conflicts and confusion
15-day deadline	Achievable	Risky — architecture eats into feature time

Why MVVM is Best for Turnly

1. **Firebase does the heavy lifting**— Firestore already provides real-time streams, caching, and offline support. Clean Architecture would force you to write abstract Repository interfaces and Data Source classes just to wrap what Firebase already gives you — pointless duplication.
2. **Speed**— With MVVM, you write a ViewModel that calls a Service, and you're done. With Clean Architecture, a single feature requires: Entity → Use Case → Repository Interface → Repository Implementation → Data Source → DTO → Mapper. That's 7 files for what MVVM does in 2.
3. **Reactive by nature**— Flutter's StreamBuilder + Firestore's real-time snapshots + GetX fit MVVM perfectly. The ViewModel exposes streams (or Rx variables), the View listens. No extra wiring needed.
4. **Explainable**— In a final project demo, you can clearly explain: "View shows UI → ViewModel handles logic → Model holds data → Service talks to Firebase." Clean Architecture is harder to justify to evaluators unfamiliar with it.

When Clean Architecture IS Better (Not This Project)

- Large teams (10+ developers) needing strict boundaries
- Apps with multiple data sources (REST API + local DB + GraphQL)
- Long-term enterprise apps maintained for years
- Projects where swapping the backend is expected

State Management: GetX

- ViewModels extend GetxController and live in the viewmodels/ folder; they call the Services
- Reactive state uses Rx variables (.obs) and Obx widgets, so the UI updates automatically when a token, queue position or status changes
- Firestore real-time streams are bound to Rx variables inside ViewModels for live queue tracking
- Dependency injection and routing also use GetX — Services and ViewModels are registered with Get.put / Get.lazyPut, and GetPage handles navigation

Example Data Flow: Booking a Token

1. Patient taps "Book Token" in the booking view → calls BookingViewModel
2. BookingViewModel calls CloudFunctionService
3. CloudFunctionService calls the Cloud Function that assigns the next token number safely
4. The result is converted into a TokenModel
5. BookingViewModel updates its Rx state and the View shows the new token number

MVVM Folder Structure for Turnly

```
lib/
├── models/            → Data classes
│   ├── user_model.dart
│   ├── hospital_model.dart
│   ├── doctor_model.dart
│   ├── department_model.dart
│   ├── token_model.dart
│   └── family_member_model.dart
│
├── services/          → Firebase communication
│   ├── auth_service.dart
│   ├── firestore_service.dart
│   ├── notification_service.dart
│   └── cloud_function_service.dart
│
├── viewmodels/        → Business logic & state (GetxController)
│   ├── auth_viewmodel.dart
│   ├── hospital_viewmodel.dart
│   ├── booking_viewmodel.dart
│   ├── queue_viewmodel.dart
│   ├── admin_viewmodel.dart
│   └── manager_viewmodel.dart
│
├── views/             → UI screens
│   ├── auth/
│   ├── patient/
│   ├── manager/
│   └── admin/
│
├── widgets/           → Reusable UI components
│   ├── hospital_card.dart
│   ├── token_tile.dart
│   └── queue_status_badge.dart
│
└── utils/             → Constants, routes, bindings, helpers
    ├── constants.dart
    ├── routes.dart
    ├── app_bindings.dart
    └── helpers.dart
```

TIP

6 folders. That's it. Each layer has a single clear job. No over-engineering, no unnecessary abstractions. Build features, not folder structures.